

Spring Framework Application Flow

1. Introduction

A typical Spring Framework web application follows a **layered architecture**.

The request normally moves through:

```
Client
↓
DispatcherServlet
↓
Controller
↓
Service
↓
Repository
↓
Database
```

After processing, the response comes back in the reverse direction:

```
Database
↓
Repository
↓
Service
↓
Controller
↓
Client
```

This layered approach provides:

- Separation of concerns
- Loose coupling
- Better maintainability
- Easier testing
- Clear responsibility for each layer

2. Overall Application Flow

Consider this request:

```
GET /employees/101
```

The flow is:

```
Client
  ↓
HTTP Request
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
HTTP Response
  ↓
Client
```

3. Client Layer

The client is the application that sends a request to the Spring application.

Examples:

- Web Browser
- Angular application
- React application

- Mobile application
- Postman
- Another microservice

Example request:

```
GET /employees/101
```

or:

```
POST /employees
```

Request body:

```
{  
  "name": "Anand",  
  "email": "anand@example.com"  
}
```

The client communicates with the server using HTTP.

4. DispatcherServlet

`DispatcherServlet` is one of the most important components in Spring MVC.

It acts as the:

Front Controller

It receives incoming HTTP requests before they reach application controllers.

Flow:

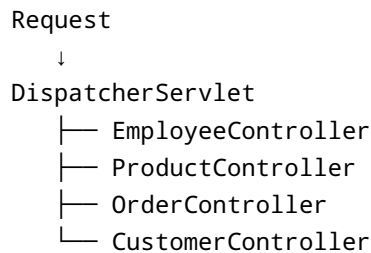
```
Client  
↓  
DispatcherServlet  
↓  
Controller
```

Its responsibilities include:

- Receiving HTTP requests
- Finding the appropriate controller
- Routing the request
- Coordinating request processing
- Working with Spring MVC components
- Returning the final response

5. Why DispatcherServlet Is Called Front Controller

Instead of every controller directly receiving requests from the web server, Spring uses one central entry point.



This centralizes request handling.

6. Controller Layer

The controller handles incoming HTTP requests.

Common annotations:

`@Controller`

and:

`@RestController`

Example:

```

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping("/{id}")
    public EmployeeResponseDto getEmployee(
        @PathVariable Long id) {

        return employeeService.getEmployee(id);
    }
}

```

Controller responsibilities:

- Receive requests
- Read path variables
- Read request parameters
- Read request bodies
- Perform request-level validation
- Call service layer
- Return response

7. Controller Should Not Contain Business Logic

Avoid putting business logic directly inside controllers.

Bad example:

```

@GetMapping("/{id}")
public Employee getEmployee(@PathVariable Long id) {

    Employee employee = repository.findById(id).orElseThrow();

    // business logic
}

```

```
        return employee;
    }
```

Better:

```
@GetMapping("/{id}")
public EmployeeResponseDto getEmployee(
    @PathVariable Long id) {

    return employeeService.getEmployee(id);
}
```

The controller should mainly coordinate HTTP requests and responses.

8. Service Layer

The service layer contains the application's business logic.

Common annotation:

```
@Service
```

Example:

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }

    public EmployeeResponseDto getEmployee(Long id) {

        Employee employee =
            employeeRepository.findById(id)
                .orElseThrow(
                    () -> new RuntimeException(
                        "Employee not found"
                    )
                )
    }
}
```

```
        );  
    };  
  
    return new EmployeeResponseDto(  
        employee.getId(),  
        employee.getName(),  
        employee.getEmail()  
    );  
}  
}
```

9. Responsibilities of Service Layer

The service layer typically handles:

- Business logic
- Business validations
- Calculations
- Transaction boundaries
- Coordination between repositories
- Calling external services
- Mapping entities to DTOs
- Applying business rules

Example business rule:

Employee salary cannot be negative

or:

Email must be unique

These rules usually belong in the service layer.

10. Repository Layer

The repository layer communicates with the database.

Common annotation:

```
@Repository
```

With Spring Data JPA:

```
public interface EmployeeRepository  
    extends JpaRepository<Employee, Long> {  
}
```

Spring Data JPA provides methods such as:

```
save()  
findById()  
findAll()  
deleteById()  
existsById()  
count()
```

11. Repository Layer Responsibilities

The repository layer handles:

- Database communication
- CRUD operations
- Database queries
- Persistence operations
- Entity retrieval
- Entity deletion

Flow:

```
Service  
  ↓  
Repository  
  ↓  
Database
```


12. Database Layer

The database stores application data.

Common databases:

- PostgreSQL
- MySQL
- Oracle
- SQL Server
- MongoDB

Example table:

```
employees
-----
id
name
email
department
```

Example:

```
SELECT *
FROM employees
WHERE id = 101;
```

13. Complete Request Flow Example

Suppose the user requests:

```
GET /employees/101
```

Step 1

Client sends request:

```
GET /employees/101
```

Step 2

`DispatcherServlet` receives it.

Step 3

Spring identifies the matching controller method:

```
@GetMapping("/{id}")
```

Step 4

Controller calls:

```
employeeService.getEmployee(101L);
```

Step 5

Service calls:

```
employeeRepository.findById(101L);
```

Step 6

Repository executes query against database.

Step 7

Database returns the employee data.

Step 8

Repository returns entity to service.

Step 9

Service applies business logic and maps entity to DTO.

Step 10

Controller returns response.

Step 11

Spring converts Java object to JSON.

Step 12

Client receives:

```
{
  "id": 101,
  "name": "Anand",
  "email": "anand@example.com"
}
```

14. Response Flow

The response travels back like this:

```
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
DispatcherServlet
  ↓
Client
```

In REST applications, Spring commonly converts Java objects into JSON.

15. HTTP Response

Example:

```
@GetMapping("/{id}")
public ResponseEntity<EmployeeResponseDto> getEmployee(
    @PathVariable Long id) {
```

```
EmployeeResponseDto employee =  
    employeeService.getEmployee(id);  
  
    return ResponseEntity.ok(employee);  
}
```

Response:

```
HTTP/1.1 200 OK
```

```
{  
  "id": 101,  
  "name": "Anand",  
  "email": "anand@example.com"  
}
```

16. MVC View Resolver Flow

For traditional MVC applications, the controller may return a logical view name.

Example:

```
@Controller  
public class EmployeeController {  
  
    @GetMapping("/employees")  
    public String employees(Model model) {  
  
        model.addAttribute("employees", employeeService.findAll());  
  
        return "employees";  
    }  
}
```

Here:

```
employees
```

is a logical view name.

Spring can use a `ViewResolver` .

Flow:

```
Controller
  ↓
View Resolver
  ↓
HTML / JSP / Thymeleaf
  ↓
Browser
```

17. REST API vs MVC Application

REST Application

```
Controller
  ↓
JSON Response
  ↓
Client
```

Example:

```
@RestController
```

Response:

```
{
  "id": 101,
  "name": "Anand"
}
```

MVC Application

```
Controller
  ↓
View Resolver
```

↓
HTML Page
↓
Browser

Example:

@Controller

18. Spring Container Role

The Spring Container manages the application objects.

It creates beans such as:

EmployeeController
EmployeeService
EmployeeRepository
DataSource
TransactionManager

These objects are called:

Spring Beans

19. IoC Container

The Spring IoC Container is responsible for:

- Creating beans
- Injecting dependencies
- Managing bean lifecycle
- Configuring beans
- Providing beans when required

Conceptually:

Spring Container
|

- |— Controller Bean
- |— Service Bean
- |— Repository Bean
- |— Configuration Bean
- |— Utility Bean

20. Dependency Injection in Application Flow

Dependencies are automatically injected by Spring.

Example:

```
@RestController
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(
        EmployeeService employeeService) {

        this.employeeService = employeeService;
    }
}
```

Spring injects the `EmployeeService`.

Similarly:

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {

        this.employeeRepository = employeeRepository;
    }
}
```

Spring injects the repository.

21. Dependency Flow

```
EmployeeController
  |
  ▼
EmployeeService
  |
  ▼
EmployeeRepository
```

Spring creates and wires these dependencies automatically.

This gives:

```
Loose Coupling
  +
Better Testing
  +
Better Maintainability
```

22. Cross-Cutting Concerns

Some features apply across multiple layers.

Examples:

- Security
- Logging
- Transaction Management
- Exception Handling
- Auditing

These are called:

Cross-Cutting Concerns

23. Security Flow

Spring Security can intercept requests before they reach the controller.


```
Client
↓
Security Filters
↓
DispatcherServlet
↓
Controller
```

Security can perform:

- Authentication
- Authorization
- Token validation
- Role validation

24. Transaction Management

Transactions are normally managed in the service layer.

Example:

```
@Transactional
public void transferEmployee(...) {

}
```

Flow:

```
Service Method Starts
↓
Transaction Begins
↓
Repository Operations
↓
Success?
├── Yes ──> Commit
└── No ───> Rollback
```

25. Logging

Logging can be added across:

```
Controller
Service
Repository
```

Example:

```
log.info("Fetching employee with id {}", id);
```

AOP can also be used for centralized logging.

26. Exception Handling

Instead of handling exceptions separately in every controller, Spring allows centralized exception handling.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<String> handleEmployeeNotFound(
        EmployeeNotFoundException exception) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(exception.getMessage());
    }
}
```

Flow:

```
Repository
  ↓
Exception
  ↓
Service
```

```
↓  
Controller Advice  
↓  
HTTP Error Response
```

27. Example Error Response

```
{  
  "status": 404,  
  "message": "Employee not found",  
  "path": "/employees/101"  
}
```

28. POST Request Flow

Suppose we create an employee.

Request:

```
POST /employees
```

Body:

```
{  
  "name": "Anand",  
  "email": "anand@example.com"  
}
```

Flow:

```
Client  
↓  
DispatcherServlet  
↓  
EmployeeController  
↓  
EmployeeService
```

```
↓  
EmployeeRepository  
↓  
Database
```

29. Controller Example

```
@PostMapping  
public ResponseEntity<EmployeeResponseDto> addEmployee(  
    @RequestBody EmployeeRequestDto request) {  
  
    EmployeeResponseDto response =  
        employeeService.addEmployee(request);  
  
    return ResponseEntity  
        .status(HttpStatus.CREATED)  
        .body(response);  
}
```

30. Service Example

```
public EmployeeResponseDto addEmployee(  
    EmployeeRequestDto request) {  
  
    Employee employee = new Employee();  
  
    employee.setName(request.name());  
    employee.setEmail(request.email());  
  
    Employee savedEmployee =  
        employeeRepository.save(employee);  
  
    return new EmployeeResponseDto(  
        savedEmployee.getId(),  
        savedEmployee.getName(),  
        savedEmployee.getEmail()  
    );  
}
```

31. Repository Example

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
}
```

Spring Data JPA handles the actual persistence operation.

32. Main Layers and Responsibilities

Layer	Responsibility
Client	Sends request and receives response
DispatcherServlet	Front controller and request router
Controller	Handles HTTP request
Service	Contains business logic
Repository	Performs data access
Database	Stores data
View Resolver	Resolves views for MVC applications
Spring Container	Creates and manages beans

33. Simple Flow to Remember

Students can remember:

```
Client
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
```

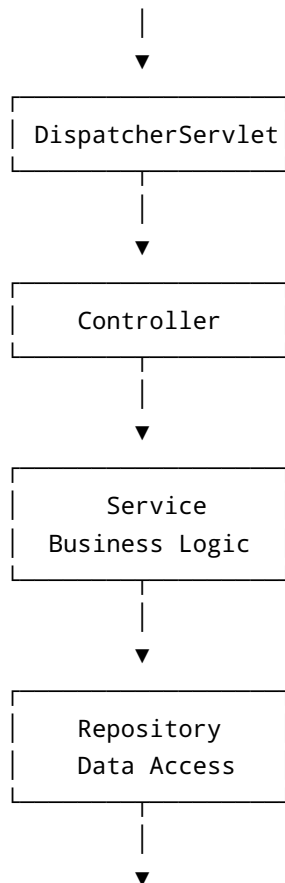
↓
Database

Response:

Database
↓
Repository
↓
Service
↓
Controller
↓
Client

34. Real-Time Architecture

Browser / Postman / Mobile





```
graph TD; subgraph Container; Database[Database]; end
```

Database

35. Why Do We Use Layers?

Separation of Concerns

Each layer performs a specific responsibility.

Loose Coupling

One layer depends on another through well-defined dependencies.

Maintainability

Changes are easier to manage.

Testability

Each layer can be tested separately.

Reusability

Business logic can be reused by different controllers.

36. Common Mistake

A common beginner mistake is:



```
graph TD; Controller[Controller] --> Repository[Repository]
```

Controller
↓
Repository

Although technically possible in very small applications, it usually mixes responsibilities.

Preferred architecture:

```
Controller
  ↓
Service
  ↓
Repository
```

37. Key Points for Students

Remember these important points:

- `DispatcherServlet` is the Front Controller.
- Controller handles HTTP requests.
- Service contains business logic.
- Repository performs database operations.
- Database stores persistent data.
- Spring Container creates and manages beans.
- Dependency Injection connects the layers.
- `@Transactional` handles transaction boundaries.
- `@ControllerAdvice` can handle exceptions globally.
- Spring Security can protect requests.
- REST applications usually return JSON.
- MVC applications may use a View Resolver.

38. Interview Answer

A good interview answer is:

In a Spring MVC application, an HTTP request first reaches the `DispatcherServlet`, which acts as the Front Controller. It identifies the appropriate controller, which then calls the service layer for business logic. The service interacts with the repository layer for database operations. The repository communicates with the database, and the result travels back through repository, service, controller, and finally to the client as an HTTP response. The Spring IoC Container manages the beans and injects the dependencies between these layers.

39. One-Line Flow

```
Client → DispatcherServlet → Controller → Service → Repository → Database → Response
```

40. Final Summary

Spring Framework application flow can be understood in three main parts:

REQUEST HANDLING

```
Client  
↓  
DispatcherServlet  
↓  
Controller
```

BUSINESS PROCESSING

```
Controller  
↓  
Service
```

DATA ACCESS

```
Service  
↓  
Repository  
↓  
Database
```

And the response comes back:

```
Database  
↓  
Repository  
↓  
Service
```

```
graph TD; A[ ] --> B[Controller]; B --> C[Client];
```

↓
Controller
↓
Client

The most important concept to remember is:

Controller handles the request, Service handles the business logic, Repository handles data access, and Spring manages the objects and dependencies connecting all these layers.